

SISTEMAS OPERATIVOS

08011442

Germán Sánchez Torres, I.S., M.Sc., Ph.D.

Profesor, Facultad de Ingeniería · Programa de Ingeniería de Sistemas

Universidad del Magdalena, Santa Marta

Teléfono: +57 605 421 4079 ext. 1138 · 301 683 6593

Edificio Docente, cubículo 3D401

Correo: gsanchez@unimagdalena.edu.co · sanchez.gt@gmail.com



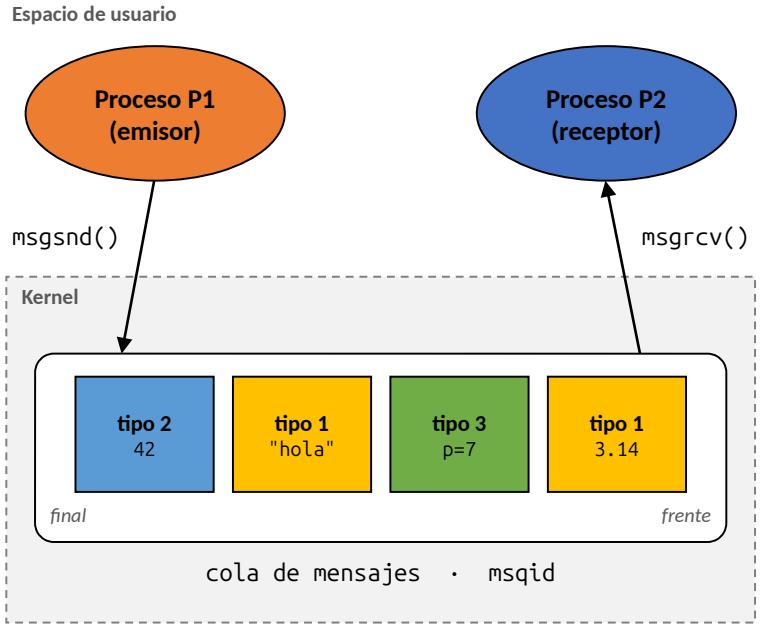
Colas de mensajes
(System V IPC)
msgget, msgsnd, msgrcv, msgctl

Contenido

1. Qué es una cola de mensajes y cómo se compara con otros mecanismos IPC
2. Ciclo de vida y estructura de un mensaje
3. Claves: ftok() e IPC_PRIVATE
4. Las llamadas msgget, msgsnd, msgrcv y msgctl
5. Ejemplos
6. Colas POSIX frente a System V
7. Resumen y ejercicios

Colas de mensajes

- Una **cola de mensajes** es una lista de mensajes que mantiene el kernel, identificada por un entero (`msqid`).
- Los procesos intercambian **mensajes completos**, con límites definidos; no un flujo de bytes como en las tuberías.
- Cada mensaje tiene un **tipo** (`long` > 0) y un cuerpo de tamaño variable.
- Sirve entre procesos **no emparentados**: la cola se localiza mediante una clave (`key_t`).
- Orden FIFO: el receptor toma el primer mensaje, o el primero de un tipo dado.
- Persiste hasta que se elimina (`msgctl IPC_RMID` o `ipcrm`) o se reinicia el sistema.



Los mensajes viven en el kernel: el emisor puede terminar y el mensaje sigue disponible para el receptor. Cada tarjeta es un mensaje con su tipo y sus datos.

Comparación de mecanismos IPC

Mecanismo	Unidad de datos	Dirección	Procesos	Sincronización
Señales	Evento: solo el número de la señal, sin datos	De un proceso a otro	Cualquiera (por pid)	Asíncrona, mediante un manejador
Tubería (pipe)	Flujo de bytes, sin límites de mensaje	Unidireccional	Emparentados: heredan los descriptores	Implícita: read bloquea si no hay datos
Memoria compartida	Bytes en una región de memoria común	Bidireccional	Cualquiera (por clave)	Explícita: semáforos o señales
Cola de mensajes	Mensajes con tipo y límites definidos	Bidireccional, multiplexada por tipos	Cualquiera (por clave)	Implícita: msgrcv bloquea si no hay mensaje

- La cola reúne **estructura** (mensajes con tipo) y **sincronización implícita** provista por el kernel.
- Costo: cada mensaje se copia dos veces (usuario → kernel → usuario); la memoria compartida evita las copias, pero exige sincronizar.

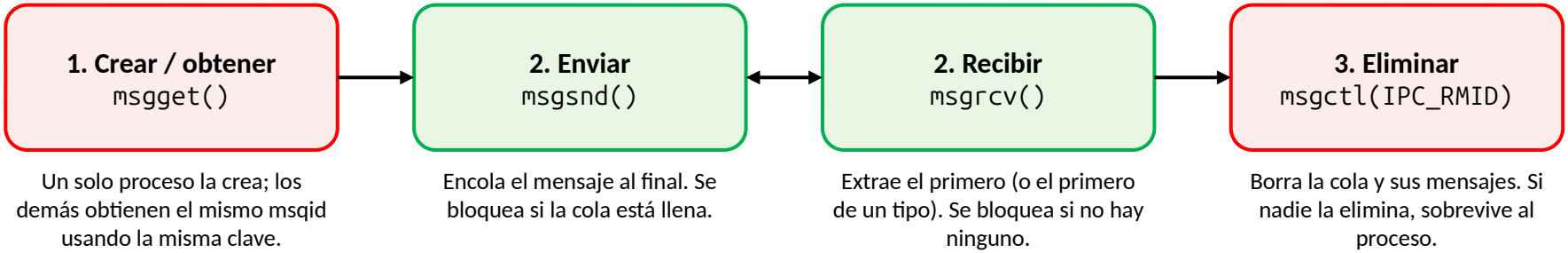
Llamadas al sistema para colas de mensajes

1. Crear u obtener una cola de mensajes → Solo un proceso
   - Crear: identificada por una clave (key) **(msgget)**
2. Operaciones sobre la cola → Todos los procesos
   - Enviar un mensaje **(msgsnd)**
   - Recibir un mensaje **(msgrcv)**
3. Control de la cola → Solo un proceso
   - Consultar, modificar o eliminar **(msgctl – IPC_RMID)**

Mismo esquema que la memoria compartida: un proceso crea el recurso, todos lo usan y uno lo elimina.

Ciclo de vida de una cola de mensajes

se repite mientras dure la comunicación, en cualquier orden



Mismo patrón que la memoria compartida (System V IPC):

`shmget()`	`shmat()` / `shmdt()`	`shmctl(IPC_RMID)`

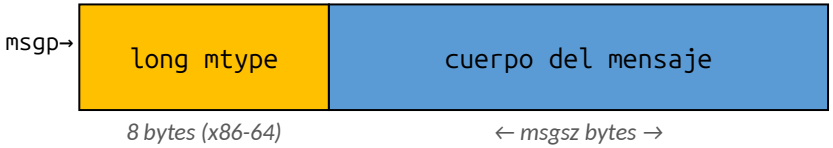
En ambos casos: obtener un identificador a partir de una clave, operar sobre el recurso y, al terminar, eliminarlo del kernel.

Estructura de un mensaje

```
struct msgbuf {
   long mtype;     /* tipo, debe ser > 0 */
   char mtext[N];  /* cuerpo del mensaje  */
};
```

- **`mtype`**: entero largo positivo que clasifica el mensaje y permite al receptor elegir cuál recibir.
- **`mtext`**: cualquier dato (texto, `struct`, arreglo). El tamaño que se pasa a `msgsnd` y `msgrcv` es solo el del cuerpo, **sin contar** `mtype`.
- Emisor y receptor deben usar la **misma estructura** para interpretar el cuerpo.
- `msgbuf` es solo una plantilla: cada programa define su propia `struct` con `long mtype` como primer campo.

Disposición en memoria del mensaje:



Ejemplos de cuerpo:

Declaración del cuerpo	msgsz al enviar
`int value;`	`sizeof(int)`
`char mtext[256];`	`strlen(mtext) + 1`
`struct data d;`	`sizeof(struct data)`

El kernel copia exactamente msgsz bytes situados justo después de mtype; la estructura debe tener al menos sizeof(long) + msgsz bytes.

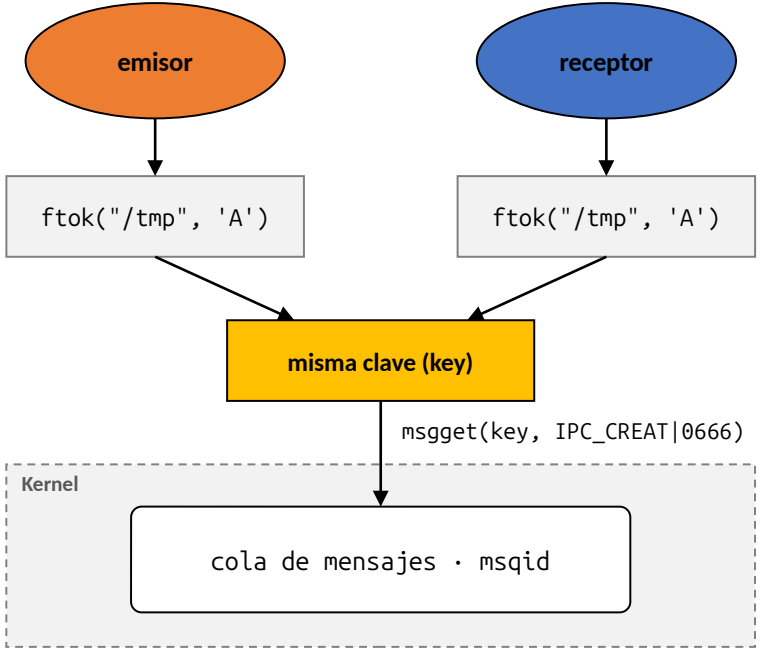
Claves: *ftok()* e *IPC_PRIVATE*

- Dos procesos independientes acceden a la misma cola si pasan la **misma clave** a `msgget`.

*key_t ftok(const char *pathname, int proj_id);*

  - Deriva la clave de un archivo existente y de un identificador de proyecto (8 bits, distinto de 0). Mismo archivo + mismo id = misma clave.

- **`IPC_PRIVATE`**: cola nueva sin clave pública; la comparten solo los procesos que heredan `msqid` con `fork`.
- Utilidades: `ipcs -q` lista las colas; `ipcrm -q msqid` elimina una.



El primero en llamar a msgget crea la cola (IPC_CREAT); el segundo obtiene el mismo msqid.

```
$ ./receptor &
$ ./emisor "Hola mundo"
[7712]enviado: Hola mundo
[7710]recibido: Hola mundo
$ ipcs -q
------ Message Queues --------
key        msqid owner    perms used-bytes messages
0x41030001 3     gsanchez 666   0          0
$ ./emisor quit          # el receptor elimina la cola
$ ipcrm -q 3             # o eliminarla a mano
```

Llamada al sistema *msgget*

int msgget(key_t key, int msgflg);

Crea una cola de mensajes nueva o devuelve el identificador de la cola asociada a la clave.

Include	`<sys/types.h>` `<sys/ipc.h>` `<sys/msg.h>`
Retorna	**Éxito:** identificador de la cola (`msqid`), un entero ≥ 0. **Falla:** -1 y `errno` indica la causa (`EEXIST`, `ENOENT`, `EACCES`, `ENOSPC`).
Argumentos	• ***key_t key***: clave de la cola: `IPC_PRIVATE` o el valor devuelto por `ftok()`. • ***int msgflg***: permisos (p. ej. `0666`) combinados con `IPC_CREAT` y, opcionalmente, `IPC_EXCL`.
Nota	La cola queda en el kernel hasta que se elimine con `msgctl(IPC_RMID)` o `ipcrm`.

msgget(): valores de los argumentos y límites

Argumento	Valor	Significado
`key`	`IPC_PRIVATE`	Cola nueva y privada: solo para procesos emparentados (fork).
	`ftok(path, id)`	Clave pública compartida entre procesos independientes.
`msgflg`	`IPC_CREAT | 0666`	Crea la cola si no existe; si existe, la abre. Los 9 bits bajos son permisos.
	`IPC_CREAT | IPC_EXCL`	Crea la cola; falla con EEXIST si ya existe (garantiza ser el creador).
	`0`	Solo accede a una cola existente; falla con ENOENT si no existe.

Límite (Linux, valores por defecto)	Valor	Parámetro del kernel
Tamaño máximo de un mensaje	8 192 bytes	`/proc/sys/kernel/msgmax`
Bytes máximos almacenados en una cola	16 384 bytes	`/proc/sys/kernel/msgmnb`
Número máximo de colas en el sistema	32 000	`/proc/sys/kernel/msgmni`

Llamada al sistema ***msgsnd***

int msgsnd(int msqid, const void *msgp, size_t msgsz, int msgflg);

Copia un mensaje desde el proceso al **final** de la cola identificada por `msqid`.

Include	`<sys/msg.h>`
Retorna	**Éxito:** `0`. Al retornar, el mensaje ya está en el kernel y el proceso puede reutilizar su buffer. **Falla:** `-1` y `errno` (`EAGAIN`, `EIDRM`, `EINTR`, `EINVAL`, `EACCES`).
Argumentos	• ***int msqid***: identificador válido de la cola. • ***const void *msgp***: puntero a la estructura del mensaje; `mtype` debe ser > `0`. • ***size_t msgsz***: tamaño del cuerpo en bytes, sin contar `mtype` (puede ser `0`; máximo `msgmax`). • ***int msgflg***: `0` → se bloquea si la cola está llena; `IPC_NOWAIT` → retorna `-1` con `EAGAIN`.

Llamada al sistema ***msgrcv***

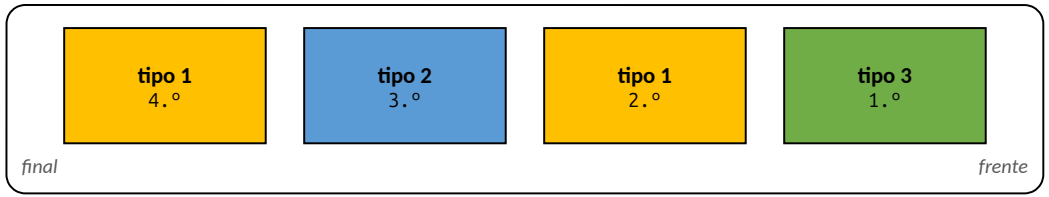
ssize_t msgrcv(int msqid, void *msgp, size_t msgsz, long msgtyp, int msgflg);

Extrae de la cola el primer mensaje que cumple el criterio de tipo y lo copia en el buffer del proceso.

Include	`<sys/msg.h>`
Retorna	**Éxito:** número de bytes copiados en el cuerpo del mensaje. **Falla:** -1 y `errno` (`ENOMSG`, `E2BIG`, `EIDRM`, `EINTR`).
Argumentos	• ***int msqid***: identificador válido de la cola. • ***void *msgp***: buffer donde se copia el mensaje (`mtype` + cuerpo). • ***size_t msgsz***: capacidad del cuerpo en el buffer del receptor. • ***long msgtyp***: criterio de selección del mensaje (siguiente diapositiva). • ***int msgflg***: `IPC_NOWAIT` → no bloquea (`ENOMSG`); `MSG_NOERROR` → trunca si el mensaje supera `msgsz` (sin él: `E2BIG` y el mensaje queda en la cola).

msgrcv(): selección por tipo (*msgtyp*)

Estado de la cola (el frente es el mensaje más antiguo):

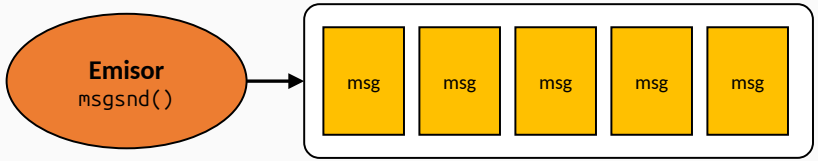


- El tipo permite **multiplexar** varios canales lógicos en una sola cola: peticiones y respuestas, un canal por cliente, prioridades.
- Si ningún mensaje coincide, `msgrcv` se bloquea o, con `IPC_NOWAIT`, retorna `-1` con `ENOMSG`.

msgtyp	Mensaje que se recibe	Con la cola del ejemplo
`0`	El primer mensaje de la cola, sin importar su tipo.	tipo 3 (1.º)
`> 0`	El primer mensaje cuyo tipo sea igual a msgtyp.	`msgtyp = 2` → tipo 2 (3.º)
`< 0`	El primer mensaje con el menor tipo que sea ≤ |msgtyp|.	`msgtyp = -2` → tipo 1 (2.º)
`> 0` con `MSG_EXCEPT` (Linux)	El primer mensaje cuyo tipo sea distinto de msgtyp.	`msgtyp = 3` → tipo 1 (2.º)

Bloqueo en *msgsnd* y *msgrcv*

Cola llena → msgsnd() se bloquea



msg_qbytes alcanzado (16 384 bytes por defecto)

- El emisor espera hasta que un receptor libere espacio en la cola.
- Con `IPC_NOWAIT`: no espera; retorna -1 y `errno = EAGAIN`.
- Con `msgsz > msgmax`: nunca cabe; retorna -1 con `EINVAL`.

Sin mensaje adecuado → msgrcv() se bloquea



- El receptor espera hasta que llegue un mensaje que coincida con `msgtyp`.
- Con `IPC_NOWAIT`: no espera; retorna -1 y `errno = ENOMSG`.
- Al llegar el mensaje, el kernel lo despierta y le entrega los datos.

- El kernel despierta al proceso bloqueado por sí solo: **no hay espera activa** (compare con `do{}while(*ptr==0)` en memoria compartida).
- Si la cola se elimina mientras un proceso espera → -1 con `EIDRM`. Si llega una señal → -1 con `EINTR`.

Llamada al sistema *msgctl*

int msgctl(int msqid, int cmd, struct msqid_ds *buf);

Consulta, modifica o elimina la cola y su estructura de control en el kernel.

Include	`<sys/msg.h>`
Retorna	**Éxito:** 0. **Falla:** -1 y `errno` (`EIDRM`, `EINVAL`, `EPERM`, `EACCES`).
Argumentos	• ***int msqid***: identificador válido de la cola. • ***int cmd***: operación a realizar: `IPC_STAT`, `IPC_SET` o `IPC_RMID`. • ***struct msqid_ds *buf***: estructura de control que se lee o escribe (`NULL` con `IPC_RMID`).

```
struct msqid_ds {
  struct ipc_perm msg_perm;  /* permisos (uid, gid, mode)  */
  time_t    msg_stime;       /* hora del último msgsnd     */
  time_t    msg_rtime;       /* hora del último msgrcv     */
  time_t    msg_ctime;       /* hora del último cambio     */
  msgqnum_t msg_qnum;        /* mensajes en la cola        */
  msglen_t  msg_qbytes;      /* bytes máximos en la cola   */
  pid_t     msg_lspid;       /* pid del último emisor      */
  pid_t     msg_lrpid;       /* pid del último receptor    */
};
```

Estructura de control de la cola. Los campos más útiles en clase: msg_qnum (cuántos mensajes esperan), msg_qbytes (capacidad) y msg_lspid / msg_lrpid (quién envió y recibió por última vez).

Operaciones de *msgctl()*

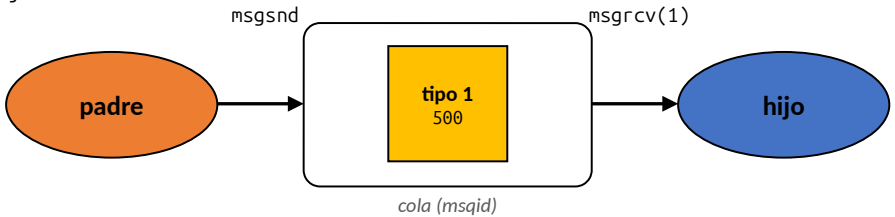
cmd	Efecto	Observaciones
IPC_STAT	Copia en buf la estructura msqid_ds de la cola: permisos, número de mensajes, capacidad, pids del último emisor y receptor.	Solo requiere permiso de lectura sobre la cola.
IPC_SET	Escribe en la cola los campos msg_perm.uid, msg_perm.gid, msg_perm.mode y msg_qbytes tomados de buf.	Solo el propietario, el creador o un proceso privilegiado. Subir msg_qbytes por encima de msgmnb exige privilegios.
IPC_RMID	Elimina de inmediato la cola y todos sus mensajes; los procesos bloqueados en msgsnd/msgrcv despiertan con EIDRM.	La cola no desaparece al terminar el proceso: eliminarla es responsabilidad del programador (o usar ipcrm).

```
int msq_id = msgget(IPC_PRIVATE, IPC_CREAT | 0600);
if(msq_id == -1){ perror("msgget"); exit(1); }
struct msqid_ds info;
if(msgctl(msq_id, IPC_STAT, &info) == -1) perror("msgctl");
printf("mensajes en cola: %lu\n", info.msg_qnum);
printf("bytes max. en cola: %lu\n", info.msg_qbytes);
msgctl(msq_id, IPC_RMID, 0);
```

Ejemplo: IPC_STAT permite saber cuántos mensajes esperan en la cola (msg_qnum) y su capacidad (msg_qbytes = msgmnb).

Ejemplo 1: el padre envía un entero al hijo

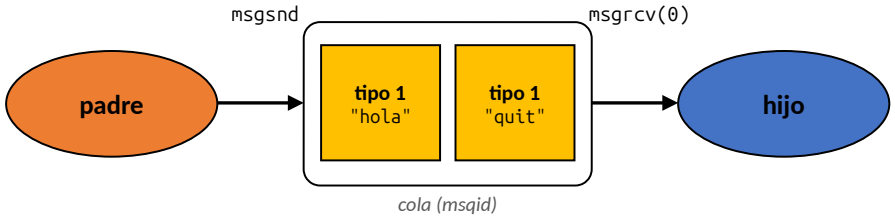
```c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>
#include <sys/ipc.h>
#include <sys/msg.h>
struct msg{
  long mtype;
  int value;
};
int main(void){
  int msq_id;
  struct msg m;
  msq_id = msgget(IPC_PRIVATE, IPC_CREAT | 0600);
  if(msq_id == -1){ perror("msgget"); exit(1); }
  if(!fork()){
    printf("[%d]child process started\n", getpid());
    msgrcv(msq_id, &m, sizeof(int), 1, 0);
    printf("[%d]msg_value->%d\n", getpid(), m.value);
  }
  else{
    printf("[%d]parent process started\n", getpid());
    sleep(2);
    m.mtype = 1;
    m.value = 500;
    msgsnd(msq_id, &m, sizeof(int), 0);
    printf("[%d]msg_value->%d\n", getpid(), m.value);
    wait(NULL);
    msgctl(msq_id, IPC_RMID, 0);
  }
  return EXIT_SUCCESS;
}
```



A diferencia de la memoria compartida (`do{}while(*ptr==0)`), `msgrcv` bloquea al hijo hasta que llega el mensaje: no hay espera activa ni hace falta una señal.

Ejemplo 2: mensajes de texto hasta "quit"

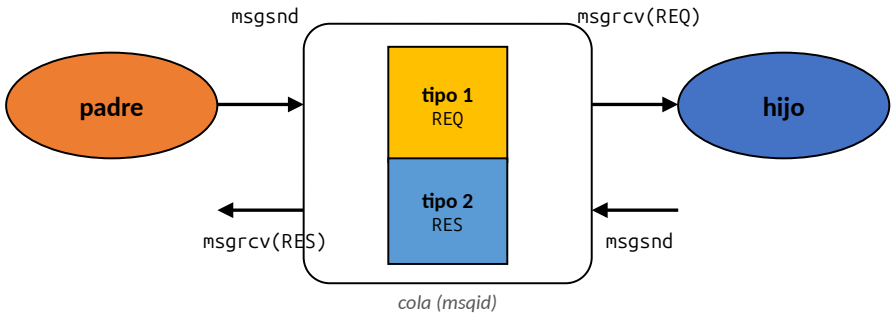
```c
#include <stdio.h>
#include <string.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>
#include <sys/ipc.h>
#include <sys/msg.h>
#define MAX_TEXT 256
struct msg{ long mtype; char mtext[MAX_TEXT]; };
int main(void){
  int msq_id, n;
  struct msg m;
  msq_id = msgget(IPC_PRIVATE, IPC_CREAT | 0600);
  if(msq_id == -1){ perror("msgget"); exit(1); }
  if(!fork()){
    printf("[%d]child process started\n", getpid());
    do{
      n = msgrcv(msq_id, &m, MAX_TEXT, 0, 0);
      if(n == -1){ perror("msgrcv"); break; }
      printf("[%d]read<--:%s [%d B]\n",getpid(),m.mtext,n);
    }while(strcmp(m.mtext, "quit") != 0);
  }
  else{
    printf("[%d]parent process started\n", getpid());
    printf("write quit to exit\n");
    m.mtype = 1;
    do{
      if(fgets(m.mtext, MAX_TEXT, stdin) == NULL)
        strcpy(m.mtext, "quit");
      m.mtext[strcspn(m.mtext, "\n")] = '\0';
      printf("[%d]write-->:%s\n", getpid(), m.mtext);
      msgsnd(msq_id, &m, strlen(m.mtext)+1, 0);
    }while(strcmp(m.mtext, "quit") != 0);
    wait(NULL);
    msgctl(msq_id, IPC_RMID, 0);
  }
  return EXIT_SUCCESS;
}
```



`msgtyp = 0` recibe cualquier tipo. Se envían `strlen + 1` bytes para incluir el terminador `'\0'`; `msgrcv` retorna cuántos bytes llegaron.

Ejemplo 3: `struct` y dos tipos de mensaje

```c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>
#include <sys/ipc.h>
#include <sys/msg.h>
#define REQ 1   /* padre -> hijo */
#define RES 2   /* hijo -> padre */
struct data{ int a; float b; };
struct msg{ long mtype; struct data d; };
int main(void){
  int msq_id;
  struct msg m;
  msq_id = msgget(IPC_PRIVATE, IPC_CREAT | 0600);
  if(msq_id == -1){ perror("msgget"); exit(1); }
  if(!fork()){
    msgrcv(msq_id, &m, sizeof(struct data), REQ, 0);
    printf("[%d]read<--[%d|%.2f]\n",getpid(),m.d.a,m.d.b);
    m.mtype = RES;
    m.d.a = m.d.a * 2;
    m.d.b = m.d.b * 2;
    printf("[%d]write-->[%d|%.2f]\n",getpid(),m.d.a,m.d.b);
    msgsnd(msq_id, &m, sizeof(struct data), 0);
  }
  else{
    m.mtype = REQ;
    m.d.a = 10;
    m.d.b = 2.3;
    printf("[%d]write-->[%d|%.2f]\n",getpid(),m.d.a,m.d.b);
    msgsnd(msq_id, &m, sizeof(struct data), 0);
    msgrcv(msq_id, &m, sizeof(struct data), RES, 0);
    printf("[%d]read<--[%d|%.2f]\n",getpid(),m.d.a,m.d.b);
    wait(NULL);
    msgctl(msq_id, IPC_RMID, 0);
  }
  return EXIT_SUCCESS;
}
```



Una sola cola sirve para ambos sentidos: cada proceso recibe únicamente los mensajes de su tipo (REQ o RES), así nunca consume su propio mensaje.

Ejemplo 4: procesos independientes con *ftok()*

emisor.c

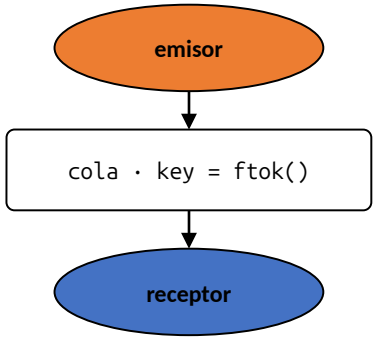
```c
#include <stdio.h>
#include <string.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/ipc.h>
#include <sys/msg.h>
#define MAX_TEXT 128
struct msg{
  long mtype;
  char mtext[MAX_TEXT];
};
int main(int argc, char *argv[]){
  key_t key = ftok("/tmp", 'A');
  int msq_id = msgget(key, IPC_CREAT | 0666);
  if(msq_id == -1){ perror("msgget"); exit(1); }
  struct msg m;
  m.mtype = 1;
  const char *t = argc > 1 ? argv[1] : "hola";
  snprintf(m.mtext, MAX_TEXT, "%s", t);
  msgsnd(msq_id, &m, strlen(m.mtext)+1, 0);
  printf("[%d]enviado: %s\n",getpid(),m.mtext);
  return EXIT_SUCCESS;
}
```

receptor.c

```c
#include <stdio.h>
#include <string.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/ipc.h>
#include <sys/msg.h>
#define MAX_TEXT 128
struct msg{
  long mtype;
  char mtext[MAX_TEXT];
};
int main(void){
  key_t key = ftok("/tmp", 'A');
  int msq_id = msgget(key, IPC_CREAT | 0666);
  if(msq_id == -1){ perror("msgget"); exit(1); }
  struct msg m;
  do{
    msgrcv(msq_id, &m, MAX_TEXT, 0, 0);
    printf("[%d]recibido: %s\n",getpid(),m.mtext);
  }while(strcmp(m.mtext, "quit") != 0);
  msgctl(msq_id, IPC_RMID, 0);
  return EXIT_SUCCESS;
}
```

Terminal

```
$ gcc -o receptor receptor.c
$ gcc -o emisor emisor.c
$ ./receptor &
$ ./emisor "Hola mundo"
[7712]enviado: Hola mundo
[7710]recibido: Hola mundo
$ ./emisor quit
[7715]enviado: quit
[7710]recibido: quit
```



Ambos programas derivan la misma clave con `ftok("/tmp", 'A')`; el primero en ejecutarse crea la cola (`IPC_CREAT`) y el otro la reutiliza. El receptor la elimina al recibir "quit".

Colas de mensajes POSIX frente a System V

Aspecto	System V (este curso)	POSIX
Cabecera / enlace	`<sys/msg.h>`	`<mqueue.h>`, enlazar con `-lrt` (glibc < 2.34)
Identificación	Clave `key_t` (ftok) e identificador `msqid`	Nombre tipo ruta (`"/micola"`) y descriptor `mqd_t`
Crear / abrir	`msgget(key, flags)`	`mq_open(name, oflag, mode, attr)`
Enviar	`msgsnd(id, msgp, msgsz, flags)`	`mq_send(mq, ptr, len, prio)`
Recibir	`msgrcv(id, msgp, msgsz, msgtyp, flags)`	`mq_receive(mq, ptr, len, &prio)`
Selección del mensaje	Por tipo (mtype): FIFO dentro de cada tipo	Por prioridad: primero el de mayor prioridad
Eliminar	`msgctl(id, IPC_RMID, NULL)`	`mq_close(mq)` + `mq_unlink(name)`
Extras	`ipcs -q`, `ipcrm -q`	Notificación asíncrona (`mq_notify`), uso con `select`/`poll`, visible en `/dev/mqueue`

Misma idea (mensajes con límites que viven en el kernel); cambia la forma de nombrar la cola y el criterio de selección.

Resumen

- Una **cola de mensajes** es un recurso del kernel identificado por `msqid` y localizado por una clave (`ftok` o `IPC_PRIVATE`).
- Los mensajes son unidades discretas con **tipo** y cuerpo; el tipo permite multiplexar canales y recibir selectivamente.
- API System V: `msgget` (crear u obtener), `msgsnd` (enviar), `msgrcv` (recibir), `msgctl` (consultar, modificar, eliminar).
- `msgsnd` y `msgrcv` bloquean por defecto: la sincronización la aporta el kernel; `IPC_NOWAIT` la desactiva.
- El tamaño que se pasa es solo el del cuerpo (`msgsz`), nunca incluye `mtype`.
- La cola sobrevive a los procesos: eliminarla siempre con `IPC_RMID` (o `ipcrm -q`).

Ejercicios

- **Ejercicio 1.** El padre envía N números al hijo por una cola (tipo 1) y el hijo responde con la suma en un mensaje de tipo 2. El padre imprime el resultado y elimina la cola.
- **Ejercicio 2.** Chat entre dos procesos independientes (dos terminales) con una sola cola obtenida con `ftok`: cada proceso lee los mensajes de su propio tipo y escribe con el tipo del otro. Terminar con "quit".
- **Ejercicio 3.** Un servidor recibe peticiones de tipo 1 que contienen el pid del cliente y responde con un mensaje cuyo tipo es ese pid. Probar con varios clientes simultáneos y explicar por qué cada cliente recibe solo su respuesta.
- **Ejercicio 4.** Repetir el ejemplo del arreglo de 10 enteros de memoria compartida usando una cola de mensajes. ¿Sigue haciendo falta la señal `SIGUSR1`? Justificar.
- **Ejercicio 5.** Enviar mensajes sin que nadie los reciba hasta superar `msgmnb` y observar con `ipcs -q` qué ocurre. ¿Qué cambia con `IPC_NOWAIT`?

Referencias y lecturas

- Kerrisk, M. *The Linux Programming Interface*, cap. 46: System V Message Queues. No Starch Press, 2010.
- Stevens, W. R. *UNIX Network Programming*, *vol. 2: Interprocess Communications*, cap. 6. Prentice Hall, 1999.
- Carretero, J. et al. *Sistemas operativos: una visión aplicada*, servicios de comunicación entre procesos. McGraw-Hill, 2007.
- Páginas de manual: `man 2 msgget`, `man 2 msgsnd`, `man 2 msgctl`, `man 3 ftok`, `man 7 sysvipc`, `man 7 mq_overview`.